

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	N. Soikanth	Reg. No.:	192411139
Section / Batch:	CSE	Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONCEPT MAPPING

Q1. Consider the concept map: Algorithm → Growth Function → Big-Omega → Lower Bound → Best-case Explain how lower bound represents minimum execution time. Extend the map with performance guarantee node. Justify why best-case analysis is useful. Interpret how this contributes to overall performance understanding.

Q2. Consider the concept map: Recurrence Relation → Substitution Method → Expansion → Simplification → Final Complexity Trace the step-by-step transformation of recurrence into a closed form. Extend the map with Guess and Verification nodes. Justify how substitution helps solve non-standard recurrences. Interpret how this structured approach improves understanding.

Q.1)

Concept map: Tight Bound (Big-O, Big-omega, Big-Theta)

Big-O(θ)

Upper Bound

Represents the maximum growth rate of an algorithm

$$T(n) \leq c_1 g(n) \text{ for all } n \geq n_0$$

It gives an asymptotic upper limit.

Big-O leads towards Big

Why Big-Theta is Preferred.

- * It gives the exact asymptotic growth rate
- * provides both upper and lower limits
- * more accurate than Big-O.
- * use for comparing algorithms when the exact performance is known.

Big-Theta(θ)

Tight Bound

Represent the exact growth rate of an algorithm.

$$c_1 g(n) \leq T(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

Interpretation of the map.

Notation meaning Nature

Big-O(θ) upper bound Approximate

Big-omega(ω) lower bound Approximate

Big-Theta(θ) Tight bound precise

Big-omega(ω)

Lower Bound

Represents the minimum growth rate of an algorithm

$$T(n) \geq c_2 g(n) \text{ for all } n \geq n_0$$

Big-omega leads towards Big-Theta

Approximate vs precise

Approximate Description

- * Describe a range of possible growth
- * Do not pinpoint exact rate
- * gives a complete picture of the algorithm

influence of parameters

- * a (number of subproblems).
- * Determines how many recursive calls are made.
- * Larger $a \rightarrow$ more total work.
- $\rightarrow$ Higher the complexity.

columns, as number of subproblems
 $b =$ factor of size reduction.
 $n \log b =$ time to divide and
 combine solutions.

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

recurrence.

and Master algorithm satisfies it.

Its running time $T(n)$ of a divide

Recurrence Relation.

Number of subproblems

Problem size reduction

Divide the problem
 Break the problem of
 size n into smaller
 subproblems

2. Solve recursively
 Recursively solve all the
 subproblems of reduced
 size

3. Combine results
 Combine the solutions.
 of subproblems to get
 the final solution.

Divide and Conquer